

Collab Docs Platform

Gerçek zamanlı ortak doküman düzenleme sistemi üzerinden dokuz backend sistem tasarımı kavramı

6 Aylık Proje Planı — Ekim 2026 / Mart 2027

Hazırlayan	Deniz Doğan
Tarih	16 Eylül 2026
Durum	Planlama tamamlandı, Ay 1 hazırlık çalışmaları başladı
Depo	<code>~/collab-docs-platform</code> (kaynak doğruluk: <code>README.md</code>)
Teknoloji	Go 1.26, Echo v4, MySQL 8, Redis 7, gorilla/websocket, Docker Compose

İçindekiler

1. Projenin amacı ve arka planı
2. Dokuz kavram ve sistemdeki yerleri
3. Neden ortak doküman editörü
4. Hedef mimari
5. Veri modeli
6. API yüzeyi
7. Çakışma çözümü: Operational Transformation (OT)
8. Aylık plan (Ay 1 – Ay 6)
9. Ay sonu ritüeli ve teslim standardı
10. Taşınan mühendislik kuralları
11. Bilinen ödünler ve kapsam dışı konular
12. Riskler ve önlemler
13. Mevcut durum ve sonraki adımlar

1. Projenin amacı ve arka planı

Bu proje, backend sistem tasarımı kavramlarını çalışan bir sistem üzerinden öğrenmeyi amaçlayan üç projelik serinin üçüncüsüdür. Her ay bir kavram kümesi ele alınır ve ay sonunda çalışan, doğrulanabilir bir çıktı üretilir. Önceki iki proje kavramları aralarında paylaşmıştı; bu proje ise kaynak dokümandaki **dokuz kavramın tamamını tek bir sistemde** hayata geçirir.

Proje	Dönem	Kapsanan kavramlar
notification-api	Şubat – Mayıs 2026	Rate Limiting, Auth & Authorization, Caching (Redis), Webhooks
realtime-chat-platform	Temmuz – Eylül 2026	Microservice, Reverse Proxy, WebSocket, Load Balancer, CDN
collab-docs-platform	Ekim 2026 – Mart 2027	Dokuz kavramın tamamı

Senaryo: birden fazla kullanıcının aynı metin dokümanını aynı anda düzenleyebildiği, dokümanları paylaşabildiği, olayları webhook ile dış sistemlere bildirebildiği ve yayınlanmış sayfaları bir edge cache üzerinden sunabildiği bir platform. Ürün amacı değil öğrenme amacı ön plandadır; her kavramın sistemde *doğal* bir gerekçesi vardır, hiçbirisi liste tamamlamak için eklenmemiştir.

Zaman çizelgesi: 6 ay, her ay bir kavram kümesi. Ay sonunda README'nin ilgili bölümü "planlanan"dan "yapılan"a çevrilir, doğrulama komutları gerçek çıktılarla güncellenir, `go test ./... -race` yeşil olur ve tek bir `Month N: <özet> commit`'i atılır.

2. Dokuz kavram ve sistemdeki yerleri

#	Kavram	Ay	Sistemde nerede yaşıyor
---	--------	----	-------------------------

1	Rate Limiting	4	gateway (HTTP için Redis'te kayan pencere; login brute-force koruması), doc-service (WebSocket op akışında bağlantı başına token bucket)
2	Load Balancer	3	gateway <code>internal/loadbalancer</code> : round-robin + doküman id üzerinde consistent hash, sağlık kontrolü
3	Authentication & Authorization	1	gateway (JWT, <code>user/admin</code>), doc-service (doküman başına <code>owner/editor/viewer</code> rolleri)
4	Caching (Redis)	4	doc-service: kullanıcı başına doküman listesi Redis'te, N instance arasında invalidasyon
5	Microservice	1	Dört servis, servis başına veritabanı, ortak sıvı ama ortak kod yok, kimlik doğrulama kenarda
6	Webhooks	5	notification-service (imzalı, yeniden denemeli teslimatlar), doc-service'teki transactional outbox ile beslenir
7	Proxy	1	gateway (<code>httputil.ReverseProxy</code>), sistemin tek dış giriş noktası
8	WebSocket	2	doc-service <code>/ws?doc=</code> : OT operasyon akışı, imleçler, presence
9	CDN	6	gateway edge cache, publish-service önünde: değişmez <code>/assets/:sha</code> , değişebilir <code>/p/:slug</code> + purge

3. Neden ortak doküman editörü

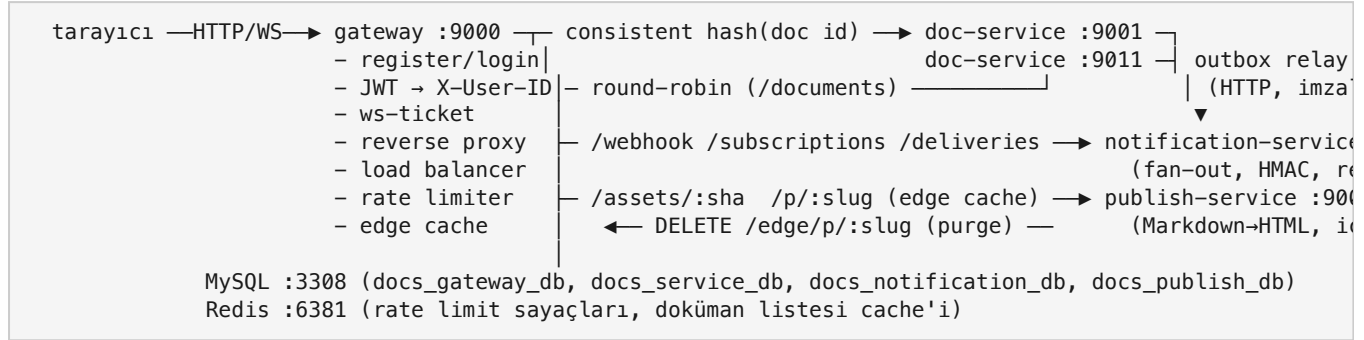
- **WebSocket zorunlu:** aynı dokümana yazan iki kişi için çift yönlü, düşük gecikmeli kanal gerekir; polling ürünü kullanılamaz kılar.
- **Load balancer + sticky routing doğruluk meselesi:** bir dokümanın operasyon günlüğü tam olarak bir süreç tarafından sıralanır. Chat projesinde odalar için "kolaylık" olan sticky session burada zorunludur; iki instance aynı dokümanı sıralarsa doküman bozulur.
- **Yetkilendirme daha zengin:** global `user/admin` yerine doküman başına roller. Geçerli bir WebSocket tutan `viewer`, edit gönderdiğinde yine reddedilmelidir.
- **Rate limiting yeni bir şekil alır:** dakikada istek değil, açık bir akışta saniyede operasyon. 5 MB'ı tek op olarak yapıştıran istemci ile 10.000 tek karakterlik op gönderen script aynı problemdir.
- **Caching gerçek bir çapraz-instance invalidasyon problemiyle karşılaşır:** üyeliği değiştiren instance, bir sonraki liste okumasını sunacak instance değildir. Bu, in-process cache yerine Redis'in gerekçesidir.
- **Webhooks tuş vuruşu hızında olay üretir:** bu da *coalescing* (birleştirme) zorunluluğu getirir. Chat projesinin "teslimat kuyruğu bellekte" ödünü transactional outbox ile yanıtlanır.
- **CDN dersinin iki yarısı da var:** içerik adresli değişmez varlıklar ve doküman başına değişebilir güzel URL, yani **purge**.
- **Reverse proxy ve microservice** iskeleti oluşturur; ilk günden itibaren kimlik doğrulama kenarda yapılır ve kimlik header'larıyla aşağıya iletilir.

Reddedilen alternatifler: canlı açık artırma (en güçlü alternatif, ancak WebSocket kullanımı daha zayıf, CDN kısmı cılız), canlı yayın sohbeti (chat projesine çok yakın), canlı konum takibi (CDN ve webhooks zorlama olurdu).

Çakışma çözümü için **karakter düzeyinde Operational Transformation (OT)** seçildi (ot.js / Google Wave soyu). CRDT reddedildi çünkü tek sıralama süreci ihtiyacını ortadan kaldırır ve böylece sticky routing dersini de kaldırır. Blok düzeyinde last-writer-wins ise fazla sığ bulundu.

4. Hedef mimari (Ay 6 sonu)

Bağımsız deploy edilebilen dört Go servisi, yerel geliştirme için bir Go workspace (`go.work`) ile bir arada. `doc-service` Ay 3'ten itibaren **N instance** olarak çalışır.



4.1 Servisler

Servis	Sorumluluk
gateway	Tek dış giriş noktası. Kullanıcılar ve kimlik doğrulama (<code>/register</code> , <code>/login</code> , <code>role claim</code> 'li JWT), tek kullanımlık WebSocket biletleri, editör sayfası, her şeyin önünde reverse proxy. JWT'yi bir kez doğrular, kimliği <code>X-User-ID</code> + <code>X-User-Role</code> olarak iletir, <code>X-Gateway-Key</code> ile isteğin kendinden geldiğini kanıtlar, <code>Authorization header</code> 'ını siler. Ay 3'ten itibaren load balancer, Ay 4'ten rate limiter, Ay 6'dan edge cache.
doc-service	Dokümanlar: metadata, üyelik (doküman başına rol), her açık doküman için OT oturumu, yalnızca ekleme yapılan operasyon günlüğü, periyodik snapshot'lar, doküman içi presence ve imleçler. Webhook'a dönüşecek outbox satırlarını yazar. Bir doküman aynı anda tam olarak bir instance tarafından sunulur.
notification-service	Webhook'lar: kullanıcının endpoint'i ve sırrı, abonelikler (hangi dokümanlar, hangi olay tipleri), teslimat günlüğü. Outbox relay'den olay alır, fan-out yapar, imzalar, yeniden denemeyle teslim eder, özel adreslere bağlanmayı reddeder.
publish-service	Belirli bir sürümdeki dokümanı statik HTML sayfasına çevirir, sayfayı ve yüklenen görselleri diskte içerik adresli saklar, doküman başına slug 'ı güncel hash'e eşler. CDN'in <i>origin</i> 'i; gateway ise edge.
presence-service yok	Chat projesi buna ihtiyaç duyuyordu çünkü bir odanın üyeleri instance'lara dağılmıştı. Burada doküman tek instance'ta yaşar; "bu dokümanda kim var" bellek içi ve kesindir. Redis bu projede yalnızca gerçekten gerektiren iki kavram içindir: caching ve rate limiting.

4.2 Portlar

Üç öğrenme projesi yan yana çalışabilsin diye seçildi (`notification-api` 8080/3306/6379, `realtime-chat-platform` 8000-8011/3307/6380 kullanır).

Bileşen	Port	Not
gateway	9000	public
doc-service	9001, 9011 (...9021)	Ay 3'ten itibaren port başına bir süreç
notification-service	9003	yalnızca gateway üzerinden
publish-service	9004	yalnızca gateway üzerinden
MySQL	3308 → 3306	container <code>docs_mysql</code>

Redis	6381 → 6379	container docs_redis
webhook alıcısı (doğrulama)	9900	geçici yerel süreç

4.3 Sırlar ve kimlerle paylaşıldığı

Her sırrın **varsayılan değeri yoktur**; ihtiyaç duyan servis, değer ayarlanmamışsa başlangıçta hemen çıkar. Öngörülebilir bir varsayılan, herkesin token sahtelemesine veya bir servisi taklit etmesine izin verirdi. Paylaşılan sır, paylaşılan kod değil: ortak Go modülü yoktur.

Sır	Paylaşan taraflar	Amaç
JWT_SECRET	yalnızca gateway	Kullanıcı token'larını imzalar ve doğrular; hiçbir backend JWT doğrulamaz
GATEWAY_SHARED_KEY	gateway → doc, notification, publish	X-User-ID 'nin gateway tarafından set edildiğini kanıtlar
NOTIFICATION_API_KEY	notification-service ← doc-service	Outbox relay'in olay göndermesi
PUBLISH_API_KEY	publish-service ← doc-service	Render istekleri

Servisler arası anahtarlar sabit zamanlı karşılaştırılır (crypto/subtle). Sağladıkları güven, ağ sınırı kadar iyidir: :9001 'e doğrudan erişebilen ve GATEWAY_SHARED_KEY 'i tutan bir çağırana istediği kullanıcı id'sini iddia edebilir. Bu, projedeki her anahtar için geçerlidir ve hiçbirinin varsayılanı olmamasının sebebidir.

4.4 Depo yapısı

collab-docs-platform/	
├ README.md	tek doğruluk kaynağı
├ go.work	use (./gateway ./doc-service ./notification-service ./publish-service)
├ docker-compose.yml	mysql:8 (3308) + redis:7 (6381); Ay 6'da servisler eklenir
├ db/	gateway_schema.sql, doc_schema.sql, notification_schema.sql (a5), publish_schema.sql
├ static/editor.html	tek dosyalık istemci, gateway tarafından / adresinde sunulur
├ gateway/	cmd/gateway, config/, internal/{controller,service,repository,domain,middleware}
├ doc-service/	cmd/doc, config/, internal/{controller,service,repository,domain,middleware}
├ notification-service/	(Ay 5) aynı katmanlama; deliverer, signer, ssrf, fanout
├ publish-service/	(Ay 6) render (goldmark), storage (içerik adresli), slug haritası

Her servis aynı dört katmanlı yapıdadır: controller → service → repository → domain; main.go 'da constructor injection ile bağlanır; database/sql ve ham SQL, ORM yok. Echo v4, gorilla/websocket, go-redis/v9, golang-jwt/v5, bcrypt.

5. Veri modeli

Şemalar kendi kendine yeterlidir (CREATE DATABASE IF NOT EXISTS + USE + IF NOT EXISTS), böylece hem canlı bir volume'a uygulanabilir hem de docker-entrypoint-initdb.d scripti olarak çalışabilir. Bu scriptler **yalnızca veri dizini boşken** çalışır; sonradan eklenen şema dosyası docker exec -i docs_mysql mysql -uroot -proot < db/<dosya>.sql ile uygulanır.

Şema	Tablo	Önemli alanlar / tasarım notu
docs_gateway_db	users	id, email (unique), password_hash, role ENUM('user','admin'), created_at

docs_service_db	documents	id, owner_id (FK yok), title, content MEDIUMTEXT (snapshot), version (content'e katlanan op sayısı), created_at/updated_at DATETIME(3); index (owner_id, updated_at)
	document_ops	Yalnızca ekleme. PRIMARY KEY (doc_id, version): aynı doküman için 42. sürümü yazmaya çalışan iki instance burada çıkarışır; sticky routing'in arkasındaki veritabanı düzeyi güvence. op JSON = [retain, "insert", -delete, ...]
	document_members	PRIMARY KEY (doc_id, user_id), role ENUM('owner', 'editor', 'viewer'). Sahip de burada bir satırdır; her yetki kontrolü tek tabloda tek sorgudur.
	outbox (Ay 5)	event_id CHAR(32) UNIQUE (idempotency anahtarı), event_type, doc_id, payload JSON, processed_at. Tarif ettiği değişikliklerle aynı transaction 'da yazılır.
docs_notification_db (Ay 5)	webhooks	user_id PK, url, secret (imzalama için düz metin)
	subscriptions	PRIMARY KEY (user_id, doc_id, event_type); event_type '*' olabilir
	deliveries	status ENUM('pending', 'delivered', 'failed'), attempts, next_attempt_at (kalıcı retry takvimi), UNIQUE (event_id, user_id)
docs_publish_db (Ay 6)	publications	doc_id PK, slug UNIQUE, version, html_hash CHAR(64). Dosyalar diskte PUBLISH_DIR/<sha256>; tabloya gerek yok, adres içeriğin kendisidir.

Gateway şeması dışındaki `user_id` sütunlarında `users` 'a **foreign key yoktur**: MySQL, farklı deployable'ların sahip olduğu şemalar arasında bütünlüğü zorlayamaz. Id, gateway'in doğruladığı token'dan geldiği için güvenilir. Eksik kısıt, sınırın bedelidir.

6. API yüzeyi

Yetki sınıfları: **public**; **user** (gateway'de Bearer JWT, aşağıya X-User-ID/X-User-Role + X-Gateway-Key olarak iletilir); **admin** (role=admin olan user); **service** (paylaşılan anahtar header'ı, gateway üzerinden asla erişilemez). Her yerde hata gövdesi `{"error": "..."}` . Üyesi olunmayan doküman 404 değil 403 döner: id'ler ardışıktır, varlığı gizlemek tiyatro olurdu.

6.1 gateway (:9000)

Metot	Yol	Yetki	Ay	Not
GET	/	public	1	static/editor.html
GET	/health	public	1	{"status": "ok"}
POST	/register	public	1	≥ 8 karakter, bir büyük harf, bir rakam → 201 {id, email, role}
POST	/login	public	1	→ {token} (24 saat, claim'ler user_id, role)
GET	/me	user	1	

POST	/ws-ticket	user	2	30 saniye, tek kullanımlık opak bilet
GET	/ws?doc=&ticket=	ticket	2	doc-service'e proxy (Ay 3'ten itibaren doc üzerinde consistent hash)
GET	/admin/users, /admin/documents	admin	1	sayfalı
GET	/health/backends	admin	3	instance sağlık görünümü
GET	/health/edge-cache	admin	6	entries, bytes, hits, misses, purges
GET	/p/:slug, /assets/:sha	public	6	edge cache; slug max-age=60 + ETag, purge edilebilir; sha immutable
DELETE	/edge/p/:slug	service	6	purge; yeniden yayında publish-service çağırır
*	/documents*, /webhook, /subscriptions, /deliveries	user	1/5	proxy

6.2 doc-service (:9001, :9011, ...)

/health dışında tüm route'lar X-Gateway-Key + X-User-ID ister. :id route'ları Ay 3'ten itibaren tek instance'a hash'lenir; /documents (liste, oluşturma) round-robin.

Metot	Yol	Gerekli rol	Ay	Not
POST	/documents	user	1	documents + document_members(owner) tek transaction'da
GET	/documents?page=&per_page=	user	1	üyesi olunan dokümanlar, updated_at DESC, id DESC ; Ay 4'ten itibaren kullanıcı başına cache
GET	/documents/:id	member	1	açıkça canlı oturumdan, değilse snapshot + op replay
PATCH	/documents/:id	owner/editor	1	yalnızca {title}; içerik WebSocket'ten değişir
DELETE	/documents/:id	owner	1	oturumu kapatır, socketleri 4004 ile düşürür
GET/PUT/DELETE	/documents/:id/members[:user_id]	member / owner	1	PUT upsert(editor viewer); DELETE o kullanıcının socketlerini 4003 ile düşürür; Ay 5'te outbox satırı
GET	/documents/:id/ops?from=	member	2	yeniden bağlanma sonrası yakalama, en fazla 500
GET	/ws?doc=	member	2	WebSocket upgrade, OT oturumu
POST	/documents/:id/publish	owner	6	publish-service'e render isteği → {slug,url,version}
POST	/documents/:id/assets	owner/editor	6	publish-service'e proxy (multipart, görsel, 5 MiB)
GET	/admin/documents	admin	1	tüm dokümanlar, sayfalı

6.3 notification-service (:9003, Ay 5) ve publish-service (:9004, Ay 6)

Servis	Metot	Yol	Yetki	Not
notification	PUT/DELETE	/webhook	user	{url} → {url, secret}; sır yalnızca burada gösterilir
notification	POST/DELETE	/subscriptions[:doc_id/:event_type]	user	event_type ∈ shared, unshared, updated, published, *
notification	GET	/deliveries	user	kullanıcının kendi denetim izi
notification	POST	/events	service X-Notification-Key	→ 202; tekrar eden event_id → 202 no-op
publish	POST	/render	service X-Publish-Key	Markdown → HTML, sha256(html) ile sakla, publications upsert, gateway'de purge
publish	POST	/assets	gateway key + user	tür header'dan değil baytlardan tespit edilir
publish	GET	/p/:slug, /assets/:sha	gateway key	origin okuması; ETag, Cache-Control, 304

7. Çakışma çözümü: Operational Transformation (OT)

Bu bölüm, kavram listesinden çıkarılamayacak tek parça olduğu için README'de tam olarak belirtilmiştir; burada özetlenir. Her şey düz metindir; zengin metin, geri alma ve çevrimdışı düzenleme tüm proje boyunca kapsam dışıdır.

7.1 Pozisyonlar ve operasyon formatı

Pozisyon ve uzunluklar **Unicode code point** cinsindendir; bayt veya UTF-16 birimi değil. Go'da []rune(s), JavaScript'te Array.from(str). Bir textarea'nın selectionStart'ı UTF-16 birimindedir ve imleç pozisyonu olarak gönderilmeden önce **dönüştürülmelidir**; aksi halde ilk emoji'de iki taraf birbirinden kopar.

Bileşen	JSON	Anlam
retain <i>n</i>	pozitif tam sayı	<i>n</i> code point'i atla
insert <i>s</i>	boş olmayan string	mevcut pozisyona <i>s</i> ekle
delete <i>n</i>	negatif tam sayı	mevcut pozisyondan <i>n</i> code point sil

```
doküman: "hello world" (11)
op:      [6, "brave ", -5, "there"]
sonuç:   "hello brave there"
```

Değişmezler (`ot.Validate` ile her gelen op'ta kontrol edilir): `baseLen(op)` uygulanacak dokümanın uzunluğuna **eşit olmalı** (uyuşmazlık → hata, asla "sığıcı uygula"); kanonik form (sıfır bileşen yok, aynı türden bitişik bileşen yok, delete-sonra-insert → insert-sonra-delete); boyut sınırı op için 64 KiB, doküman için 1 MiB code point.

7.2 Üç algoritma

- **apply(doc, op)**: bileşenleri `[] rune(doc)` üzerinde bir imleçle yürü; retain kopyalar, insert ekler, delete atlar. $O(\text{len}(\text{doc}))$.
- **compose(a, b)**: `apply(apply(d,a),b) == apply(d, compose(a,b))`. İstemci tamponlanmış yerel op'ları tek op'a katlarken, sunucu snapshotter kuyruğu snapshot'a katlarken kullanır. Rastgele op üreticiyle fuzz testi **projedeki en değerli tek testtir**.
- **transform(a, b) → (a', b')**: aynı durum üzerinde üretilmiş iki op için `apply(apply(d,a),b') == apply(apply(d,b),a')`. Aynı pozisyondaki eşzamanlı insert'lerde **günlükteki op pozisyonu kazanır**; iki taraf da günlükteki op'u `a` olarak geçirir. Bu kural kodda belgelenir ve `TestTransformConcurrentInsertLogOrderWins` ile açıkça test edilir; taraflar hangi argümanın `a` olduğunda anlaşılamazsa metinler *farklı* noktalarda yakınsar ve sistemde başka hiçbir şey nedenini söylemez.

7.3 Sunucu: doküman oturumu

Açık doküman başına bir `Session`, **tek goroutine'e** (`run()`) aittir, yalnızca kanallar üzerinden erişilir. Doküman durumu etrafında mutex yoktur; sıralama goroutine'in seri yürütmesidir. Durum: `content []rune, version, son 1000 (version, op)` çiftinden oluşan halka, `clients map[*Client]role`.

- **Açılış**: `snapshot + document_ops WHERE version > snapshot` yükle ve katla.
- **İstemci op'u** `{v, op, seq}: viewer → forbidden; v > version → 4400; version - v > 1000 → 4409` (yeniden yükle); aksi halde halkadaki `(v, version)` aralığındaki her op'a karşı transform et, doğrula, uygula, halkaya it.
- **Persist-before-ack**: writer goroutine `INSERT INTO document_ops` yapar; commit döndükten sonra gönderene `ack`, diğerlerine `op` gider. Maliyet: batch başına bir MySQL gidiş-dönüş. Kazanç: istemci, veritabanında olmayan bir op için asla ack tutmaz. Çökme yalnızca *tekrar* üretir, kayıp değil; `(doc_id, version)` primary key tekrarı hataya çevirir.
- **Snapshot**: 100 op veya 30 saniye (kırılyse). `UPDATE ... WHERE version < ?` monoton guard. Op'lar silinmez; günlük tarihtir.
- **İmleçler**: doğrulanır, fan-out yapılır, asla saklanmaz.
- **Üyelik değişikliği** oturum kanalına gelir: viewer'a düşürülme → soket kalır, op'lar reddedilir; kaldırılma → 4003.
- **Kapanış / shutdown**: son istemci ayrılınca snapshot, 60 saniye sıcak tutma; SIGTERM'de upgrade kabulü durur, kirli oturumlar snapshot'lanır, soketler 1001 ile kapanır.
- Writer oturum goroutine'ini asla bloklamaz; kuyruk dolunca oturum soketlerden okumayarak **backpressure** uygular, op düşürmez.

7.4 Wire protokolü ve istemci

Tüm frame'ler JSON metin frame'idir. İstemci → sunucu: `op` (aynı anda en fazla bir tane uçuşta), `cursor`, `ping`. Sunucu → istemci: `snapshot`, `ack`, `op`, `cursor`, `presence`, `title`, `error`. Kapanış kodları: 4003 üyelik iptal, 4004 doküman silindi, 4400 protokol ihlali, 4409 çok geride, 4429 rate limit (Ay 4), 1011 sunucu hatası, 1001 shutdown.

İstemci (`static/editor.html`, `ot.js` üç durumlu makine): `Synchronized` | `AwaitingConfirm(outstanding)` | `AwaitingWithBuffer(outstanding, buffer)`. Textarea'dan `op` üretimi: her `input` olayında önceki değerle yeni değer arasında en uzun ortak önek/sonek farkı; olay başına tam olarak bir `op`. Yeniden bağlanma: 1, 2, 4, 10 saniye backoff; `snapshot` gelince onaylanmamış yerel `op`'lar *düşürülür* ve bir satırlık uyarı gösterilir.

7.5 Limitler (varsayılan, env ile ayarlanabilir)

Ad	Varsayılan	Nerede uygulanır
<code>DOC_MAX_CODEPOINTS</code>	1 048 576	oturum, <code>targetLen</code>
<code>OP_MAX_BYTES</code>	65 536	ws read limit ve validator
<code>OP_RING_SIZE</code>	1000	oturum
<code>SNAPSHOT_EVERY_OPS / _SECONDS</code>	100 / 30	snapshotter
<code>SESSION_IDLE_SECONDS</code>	60	oturum kapanışı
<code>WS_OPS_PER_SECOND</code> (Ay 4)	20, burst 40	bağlantı başına token bucket
<code>WS_OPS_PER_MINUTE_PER_USER</code> (Ay 4)	600	Redis, bağlantılar ve instance'lar arası

8. Aylık plan

Her ay dört haftalık bir kırımla verilmiştir. Haftalar rehber niteliğindedir; ayın çıkış kriterleri bağlayıcıdır. Her ay için kapsam, tasarım notları, doğrulama ve çıkış kriterleri README ile birebirdir.

Ay 1 — Ekim 2026 — Auth & Authorization, Microservice, Reverse Proxy

Hedef: giriş yapabilen, doküman oluşturup paylaşabilen ve salt okunur açabilen iki servis ve bir tarayıcı sayfası. WebSocket henüz yok. Sonraki ayların kimlik ve sınırlar hakkında varsaydığı her şey bu ay kararlaştırılır.

Hafta	İş
1	Depo iskeleti: <code>go.work</code> , <code>docker-compose.yml</code> (MySQL 3308 + Redis 6381), <code>db/gateway_schema.sql</code> , <code>db/doc_schema.sql</code> , <code>.gitignore</code> . gateway: <code>/register</code> , <code>/login</code> , <code>/me</code> , <code>/admin/users</code> (notification-api'den port), JWT claim'leri <code>user_id</code> , <code>role</code> , <code>exp</code> (24 saat), <code>request_id</code> + <code>logger</code> middleware (loglarda yalnızca path, asla query string). (Eylül'de öne alınarak tamamlandı.)
2	İlk günden edge auth: <code>internal/proxy/trusted_proxy.go</code> (chat'ten port) Bearer token'ı doğrular, <code>X-User-ID</code> , <code>X-User-Role</code> , <code>X-Gateway-Key</code> set eder, istemciden gelen kopyaları üzerine yazar , <code>Authorization</code> 'ı siler ve proxy'ler. Admin route'ları proxy öncesi ek olarak <code>role</code> kontrol eder. <code>/documents*</code> → <code>DOC_SERVICE_URL</code> (<code>httputil.ReverseProxy</code>).
3	doc-service: <code>gateway_auth</code> middleware (sabit zamanlı anahtar karşılaştırma, <code>X-User-ID</code> parse), documents CRUD, members CRUD, <code>/admin/documents</code> , <code>updated_at</code> DESC, <code>id</code> DESC sayfalama. Yetki kontrolleri <code>document_members</code> üzerinde tek sorgu. Sahip satırı dokümanla aynı transaction'da.
4	<code>static/editor.html</code> v1: login formu, doküman listesi, oluşturma, açma (salt okunur <code>textarea</code>), sahipler için üye paneli. Testler: <code>validation</code> , <code>jwt</code> , <code>gateway_auth</code> , üye yetki matrisi (owner/editor/viewer × her endpoint) <code>httptest</code> ile; <code>DOCS_TEST_DSN</code> set edilmişse gerçek MySQL, değilse skip. README güncelleme, commit.

Tasarım notları: gateway **tek** JWT doğrulayıcıdır; backend'ler header'lara güvenir çünkü anahtar kimin set ettiğini kanıtlar. Yetki doc-service'te kontrol edilir, gateway'de değil: gateway dokümanın ne olduğunu bilmez. Ayın dersi: *kimlik doğrulama kenarda, yetkilendirme kaynağın sahibinde*.

Çıkış kriterleri: yetki matrisi testi geçer; `editor.html` giriş yapar, oluşturur, paylaşır ve açar; `Authorization` hiçbir doc-service log satırında görünmez; istemciden gönderilen kimlik header'ları atılır (`X-User-ID: 1` ile B'nin listesi değil A'nın listesi dönmez); `:9001` 'e gateway anahtarı olmadan doğrudan istek `401` alır.

Ay 2 — Kasım 2026 — WebSocket

Hedef: tek doc-service instance'ında, aynı dokümanı canlı düzenleyen iki tarayıcı. OT ayı; Bölüm 7'deki spesifikasyon sözleşmedir.

Sıralama kuralı: doc-service/internal/ot/ paketi ve fuzzer'ı **ilk ve tek başına** yapılır. Fuzzer temiz geçmeden ayın başka hiçbir işine başlanmaz.

Hafta	İş
1	internal/ot/: Operation, Validate, Normalize, Apply, Compose, Transform tablo testleriyle; rastgele op üreticiyle iki özellik testi (compose doğruluğu, TP1); el yazımı vakalar (aynı pozisyona insert vs insert, örtüşen delete'ler, silinen aralığa insert, eklenen aralığa delete); tie-break testi.
2	internal/session/: Session goroutine'i, op halkası, batching yapan writer goroutine'i, snapshotter, boşa kapanış, shutdown snapshot'ı. internal/ws/: upgrade handler (üyelik → rol; WS_ALLOWED_ORIGINS, eksik Origin kabul edilir), read/write pump'lı Client (pongWait 60 s, writeWait 10 s, SetReadLimit), yavaş tüketici eviction'ı.
3	Chat'ten bilet değişimi portu (token asla URL'de seyahat etmez; gateway bileti kullanır, kimlik header'larını set eder, ?ticket= 'i siler). GET /documents/:id/ops?from=. GET /documents/:id canlı oturum açıksa ondan cevap verir.
4	editor.html v2: durum makinesi, textarea diff'i, imleç koruma, uzak imleçler, presence listesi, backoff ile yeniden bağlanma. Ack gecikmesi ölçümü (tek istemci ve yük altında batching). README, commit.

Tasarım notları: persist-before-ack ayın büyük kararıdır; ölçülen ack gecikmesi yazılır. Neden mutex değil doküman başına bir goroutine: transform-apply-log dizisi doküman başına atomik olmalıdır ve goroutine kilidi unutmayı imkânsız kılar; ayrıca run() döngüsünü sıralamanın tanımlandığı tek yer yapar, Ay 3 buna dayanır.

Çıkış kriterleri: aynı anda yazan iki tarayıcı sekmesi her seferinde yakınsar; fuzzer'lar 10 000 iterasyon temiz geçer; kill -9 hiçbir ack'lenmiş op'u kaybetmez; 101 op sonrası documents.version = 100 ve içerik katlanmış (snapshot 100'de tetiklendi).

Ay 3 — Aralık 2026 — Load Balancer

Hedef: gateway arkasında N doc-service instance'ı; sticky routing doğruluk argümanı görünür kılınmış.

Hafta	İş
1	gateway/internal/loadbalancer/ chat'ten port: Strategy arayüzü, RoundRobin (sağlıklı liste üzerinde), ConsistentHash (150 sanal düğüm, FNV-1a + MurmurHash3 finalizer; regresyon testleri dokümanlar için yeniden adlandırılır), sağlık poller'ı (LB_HEALTH_INTERVAL_SECONDS=5 , başlangıçta bir senkron tarama), GET /health/backends (admin).
2	main.go 'da route başına strateji: GET /ws?doc= → doc üzerinde consistent hash; /documents/:id ve altındaki her şey → :id üzerinde consistent hash (aynı süreç okumaları, üyelik değişikliklerini ve yayını sunmalı); GET/POST /documents → round-robin. Bu ikinci satır chat projesinden farklıdır: bir dokümana HTTP yazımları da oturumuna ulaşmalıdır.
3	INSTANCE_ID (varsayılan doc-<port>), X-Doc-Instance cevap header'ı (101'de de), proxy atlaması boyunca paylaşılan request_id . Fail closed: halka tüm yapılandırılmış instance'lar üzerine kurulur ve asla yeniden kurulmaz; instance'ı düşük doküman geri gelene kadar 503 alır.
4	Shutdown sıralaması: SIGTERM → health endpoint hemen 503 → bir health aralığı bekle → snapshot ve soketleri kapat. Doğrulama senaryoları (4 doküman × 3 bağlantı asla karışmaz; B'nin soketi yanlış instance'ta değil doğru instance'ta 4003 ile kapanır; kill -9 sonrası hash'lenen dokümanlar 503, /documents her zaman 200). README, commit.

Çıkış kriterleri: hiçbir doküman iki instance'ta gözlemlenmez; halka testleri geçer; öldürülen instance'ın dokümanları aynı instance'ta geri gelir.

Ay 4 — Ocak 2027 — Caching (Redis) + Rate Limiting

Hedef: N instance arasında paylaşılan depo gerektiren iki kavram için Redis devreye girer. İkisi de `notification-api` 'de yapılmıştı; ikisi de burada çok-instance'lı sistemin zorladığı biçimde yeniden yapılır.

Hafta	İş
1	Caching: GET /documents için <code>doclist:<user_id></code> = ilk sayfanın JSON'u, TTL 300 s. Round-robin liste endpoint'inde read-through; write-through invalidasyon (DEL): doküman oluşturma/silme/yeniden adlandırma (sahibin ve her üyenin anahtarı), üye ekleme/çıkarma (o üyenin anahtarı) ve her snapshot'ta (liste <code>updated_at</code> 'e göre sıralı, <code>updated_at</code> snapshot'ta değişir, her op'ta değil). Boş liste de cache'lenir (negatif caching).
2	Gateway rate limiting: tüm yetkili HTTP için <code>user_id</code> (public route'larda IP) anahtarıyla Redis'te kayan pencere günlüğü , tek Lua script (ZADD/ZREMRANGEBYSCORE/ZCARD/EXPIRE atomik), 120/dk, 429 + Retry-After. POST /login için <code>email</code> + IP ile sabit pencere (notification-api portu), 5/dk; yanlış şifre ile rate-limit cevabı aynı (enumeration oracle'ı önlemek için).
3	WebSocket rate limiting: bağlantı başına bellek içi token bucket (golang.org/x/time/rate), 20/s burst 40, aşımda <code>error{rate_limited}</code> , 10 saniyede üç aşım → 4429. <code>user_id</code> başına instance'lar arası Redis INCR, 600/dk → 4429. <code>cursor</code> frame'leri 20/s üstünde sessizce düşürülür.
4	Ölçümler: sabit pencere vs kayan pencere sınır davranışı (sabit pencere sınırdan 2× limit kabul eder), her ikisinin sayıları yazılır. Redis çökmesinde limiter'lar fail open (log satırıyla), cache bypass. Doğrulama: 130 istek → 120 × 200, 10 × 429; 6 login → 5 × 401, 1 × 429; cache anahtarı diğer instance'tan invalidasyon. README, commit.

Tasarım notları: WebSocket limiter'ı neden in-process: bağlantı başımadır, bağlantı tek instance'tadır ve tuş vuruşu başına Redis gidiş-dönüşü Redis'i ack yoluna koyardı. Kullanıcı başına Redis sayacı in-process olanın göremediği durum içindir: aynı kullanıcı, instance'lar arası on bağlantı. Cache'te ders: cache'lenen görünümün *ne kadar* bayat olabileceğini seçmek ve adlandırmak; burada bir dokümanın listedeki yeri son düzenlemesinden en fazla `SNAPSHOT_EVERY_SECONDS` geride kalabilir.

Çıkış kriterleri: liste okumaları Redis'e gider; invalidasyon instance'ları aşar; dört limiter'ın hepsi gözlemlenebilir şekilde çalışır; Redis kapalıyken sistem ayakta kalır.

Ay 5 — Şubat 2027 — Webhooks

Hedef: doküman olayları kullanıcının kayıtlı endpoint'ine imzalı, yeniden denemeli ve **veritabanı yazımı ile teslimat arasında asla kaybolmadan** ulaşır. Chat projesinin `notification-service`'i temeldir; transactional outbox yeni kısımdır.

Hafta	İş
1	doc-service'te outbox: üye ekleme/çıkarma, yayın (Ay 6) ve başlık değişikliği olay satırını değişiklikle aynı transaction 'da yazar. <code>document.updated</code> farklıdır: <i>snapshotter</i> tarafından, doküman başına snapshot başına en fazla bir kez, <code>{doc_id, version, editors:[...]}</code> payload'ıyla yazılır. Coalescing budur: tuş vuruşu hızındaki düzenlemeler doküman başına 30 s / 100 op'ta en fazla bir webhook olur.
2	Outbox relay: her doc-service instance'ında bir goroutine <code>SELECT ... WHERE processed_at IS NULL ORDER BY id LIMIT 50 FOR UPDATE SKIP LOCKED</code> ile her <code>OUTBOX_POLL_MS=500</code> 'de yoklar, her birini notification-service /events'e <code>X-Notification-Key</code> ile POST eder, 202 'de <code>processed_at</code> işaretler. <code>SKIP LOCKED</code> N instance'ın aynı tabloyu çift göndermeden yoklamasını sağlar; aşağıdaki <code>event_id</code> unique key yedek güvencedir. At-least-once, idempotent kılınmış.
3	notification-service chat'ten port (<code>webhook_service</code> , <code>signer</code> , <code>ssrf</code> , <code>deliverer</code> , <code>fanout_service</code>) şu değişikliklerle: abonelikler <code>event_type</code> taşır; fan-out = <code>(doc_id, event_type) ∪ (doc_id, '*')</code> aboneleri, eyleyen hariç, endpoint'i olanlar; <code>document.shared</code> ayrıca davet edilen kişiye de gider. Kalıcı retry: <code>deliveries.next_attempt_at</code> takvimdir; worker havuzu vadesi gelen satırları (<code>SKIP LOCKED</code>) yoklar. Backoff 1, 2, 4, 8, 16 s ±%50 jitter, 5 deneme, 4xx nihai, 5xx /transport yeniden denenir. İmza: <code>X-Signature: sha256=HMAC(secret, "<X-Timestamp>.<body>")</code> . SSRF filtresi: kayıta string kontrolü, transport'ta dial-time kontrolü, loopback yalnızca <code>WEBHOOK_ALLOW_LOOPBACK=true</code> ile.
4	Gateway <code>/webhook</code> , <code>/subscriptions</code> , <code>/deliveries</code> 'i edge auth ile proxy'ler. Doğrulama: :9900'de imza doğrulayan alıcı; bir dakikada 300 karakter → ~2-3 <code>document.updated</code> ; A B'yi çıkarır → B <code>document.unshared</code> alır; alıcı 503 verirken notification-service <code>kill -9</code> → <code>next_attempt_at</code> onurlandırılır; outbox insert'ten hemen sonra doc-service <code>kill -9</code> → diğer instance'ın relay'i işler; <code>processed_at IS NULL</code> sayısı 0. README, commit.

Çıkış kriterleri: teslimat satırı olmayan olay yok; çökme boyunca kaybolan teslimat yok; coalescing ölçülmüş; SSRF vakaları reddedilmiş.

Ay 6 — Mart 2027 — CDN ve sertleştirme

Hedef: yayınlanmış dokümanlar ve görselleri, iki cache disiplini de (sonsuz dek değişmez ve purge'lı değişebilir) uygulayan bir edge cache üzerinden sunulur; proje bir durma noktasına sertleştirilir.

Hafta	İş
1	publish-service: POST /assets (içerik adresli görsel deposu, tür baytlardan tespit, 5 MiB; media-service portu), POST /render (goldmark Markdown → HTML, başlıklı minimal şablon; sha256(html) 'de saklanır; publications upsert; ardından gateway'de DELETE /edge/p/:slug), origin okumaları /p/:slug ve /assets/:sha uygun header'larla. Dokümanlarda görsel referansı Markdown ![alt](/assets/<sha>); editörün görsel butonu yükler ve ekler.
2	Gateway edge cache (internal/edgecache, chat'ten port: bayt sınırlı LRU, nesne başına üst sınır, miss'te single-flight, cache'lenemeyen cevaplar için BYPASS, X-Cache header'i) değişebilir durumla genişletilir: /assets/:sha sonsuza dek cache; /p/:slug origin'in max-age=60 'ıyla cache, süre dolunca If-None-Match ile revalidate (X-Cache: REVALIDATED); purge ile yeniden yayın sonraki istekte görünür. Purge best-effort'tur (gateway kapalı olabilir; TTL yedektir), gerçek CDN'lerin davranışı da budur. / (editor.html) aynı cache'ten dosya hash'i ETag'iyile. Cache Authorization taşıyan isteklerin ve Set-Cookie/private cevapların asla saklanmadığını test ile kanıtlar.
3	GET /health/edge-cache purges ve revalidations kazanır. POST /documents/:id/publish doküman başına 6/dk rate limit (render sistemdeki en pahalı istek). Doğrulama: aynı sayfanın sürümü başına bir origin fetch; purge gözlemlenebilir; 61 s sonra REVALIDATED; Authorization ile BYPASS.
4	Sertleştirme / durma noktası: servis başına Dockerfile ve tam docker-compose.yml (dört servis, iki doc-service replikası, MySQL, Redis) — tüm sistem tek komutla kalkar; her serviste graceful shutdown denetimi (drain sırası belgelenir); make check içinde go vet ve -race; Bilinen ödünler üzerinden son geçiş; Ay 1 duman testi uçtan uca. README, final commit.

Çıkış kriterleri: bir sayfanın sürümü başına kaç okuma olursa olsun tek origin fetch; purge gözlemlenebilir; docker compose up dokuz kavramın tamamını ayağa kaldırır.

9. Ay sonu ritüeli ve teslim standardı

1. README'nin Month N bölümü *planlanandan yapılane* yeniden yazılır; "planlanan" metin eklenmez, değiştirilir. Orijinal niyet yalnızca sonuç farklıysa ve fark dersse tutulur.
2. Doğrulama bloğu okuyucunun çalıştırabileceği gerçek çıktılarla güncellenir.
3. Bilinen ödümler listesi güncellenir.
4. `go test ./... -race` yeşil; OT özellik testleri için `-count=1000`.
5. Tek commit Month N: <özet>; pre-push inceleme panelinin bulguları `Address review findings: ...` commit'iyle giderilir, bastırılmaz.
6. `Running locally` bloğu yeni servisler eklendikçe güncellenir.

10. Taşınan mühendislik kuralları

Her biri bir önceki projede kazanıldı; kural değil gerekçe önemlidir.

- **Hiçbir sırrın varsayılana yok.** Konfigürasyonundan önce yayınlanan bir binary başlangıçta gürültülü şekilde çıkar.
- **Paylaşılan sır, paylaşılan kod değil.** Servisler arası ortak Go modülü yok; her biri ihtiyacını kopyalar. Kopyalar arası sapma bağımsız deploy edilebilirliğin dürüst bedelidir.
- **Servis başına veritabanı; şemalar arası foreign key yok.**
- **Kimlik doğrulama kenarda, yetkilendirme kaynağın sahibinde.** Bu projede yeni, Ay 1'de kararlaştırıldı.
- **Her paylaşılan anahtar için sabit zamanlı karşılaştırma** (`crypto/subtle`).
- **Tek goroutine'li sıcak döngüyü hiçbir şey bloklamaz.** Writer, outbox relay ve her dışa HTTP çağrısı sınırlı kuyruk arkasındadır. Tek bilinçli istisna: oturum, düzenlemeleri düşürmek yerine soketlere backpressure uygular.
- **Her sayfalı sıralamada `DATETIME(3)` ve `id tiebreak`.** Saniye çözünürlüklü eşitlikler `LIMIT/OFFSET` 'i belirsiz yapar.
- **`docker-entrypoint-initdb.d` yalnızca boş volume'da çalışır;** şema dosyaları canlı volume'a uygulanabilecek şekilde kendi kendine yeterlidir.
- **Her heartbeat/TTL çifti için $3 \times$ kuralı** ($\text{aralık} \times 3 \leq \text{TTL}$), iki kaçırma tolere edilir.
- **Her proxy'lenen istek iki tarafta da görünür bir `request_id` taşır** ve cevaplayan instance kendini bir header'da adlandırır.
- **Loglar path'i kaydeder, asla query string'i değil** (biletler ve token'lar daha önce query string'de bulundu).
- **WebSocket upgrade'de eksik `Origin` kabul edilir,** yanlış olan reddedilir. Cross-site WebSocket hijacking bir tarayıcı saldırısıdır ve tarayıcılar her zaman `Origin` gönderir.
- **Canlı bağımlılık gerektiren testler bağımlılık yoksa skip eder,** böylece `go test ./... -race` her yerde çalışır.
- **README, tarif ettiği kodla aynı commit'te güncellenir.**

11. Bilinen ödünler ve kapsam dışı konular

Her ay revize edilir. Baştan kararlaştırılan başlangıç listesi:

- **Yalnızca düz metin.** Zengin metin, geri al/yinele, yorum yok. Nitelikli metin üzerinde OT farklı (daha büyük) bir algoritmadır; sistem tasarımı dersleri ona ihtiyaç duymaz.
- **Çevrimdışı düzenleme yok.** Onaylanmamış yerel op'ları olan yeniden bağlanan istemci onları düşürür.
- **Ölü instance dokümanlarını beraberinde götürür**, geri gelene kadar (fail-closed sticky routing). "Gerçek" çözüm paylaşılan bir op günlüğüdür (Redis Streams veya kuyruk); bu, kılık değiştirmiş dağıtık konsensüstür ve bilinçli olarak yapılmaz.
- **Instance üyeliği statiktir** (DOC_SERVICE_URLS başlangıçta). Servis keşfi farklı bir derstir.
- **Sağlık yoklanır, gözlemlenmez**; instance ölümü ile sonraki yoklama arasındaki boşluktaki istek 502 alır.
- **Tek gateway.** Bilet deposu ve edge cache süreç başıdır.
- **Cache miss stampede ele alınmaz**; anahtar başına single-flight bilinen çözümdür.
- **Redis kapalıyken rate limiter'lar fail open**; erişilebilirlik katılıktan önce.
- **Webhook sırları düz metin saklanır** (imzalama değeri gerektirir).
- **Op günlüğü asla budanmaz.** Bu ölçekte tarih ucuzdur.
- **document.updated granülerliği snapshot aralığıdır.** Her tuş vuruşunu isteyen entegrasyon webhook değil WebSocket istemcisi olmalıdır.
- **Persist-before-ack, op batch'i başına bir veritabanı gidiş-dönüşüne mal olur.** Alternatif (önce ack, sonra asenkron persist) reddedildi çünkü veritabanının hiç almadığı bir düzenlemeyi onaylayabilir.

Önceki projelerden uygulanan dersler

Chat projesi ödünü	Burada
Üç kopya jwt.go, iki serviste hiç yok	Yapısal olarak çözüldü: Ay 1'den edge auth; hiçbir backend JWT doğrulamaz
Teslimat kuyruğu bellekte; çökme kuyruktaki teslimatları kaybeder	Çözüldü: doc-service'te transactional outbox + notification-service'te kalıcı next_attempt_at retry (Ay 5)
Edge cache'te purge yok	Genişletildi: değişmez varlıkların yanında purge ve revalidation'lı değişebilir /p/:slug (Ay 6)
Yazma frame'lerinde rate limiting yok	Çözüldü: op'larda bağlantı başına token bucket, imleçler limit üstünde düşürülür (Ay 4)
Presence eventually consistent	Atlatıldı: presence oturum başına bellek içi ve kesin, çünkü dokümanlar sticky
Ölü instance odalarını götürür; üyelik statik; sağlık yoklanır; webhook sırları düz; edge cache süreç başına	Korundu , aynı gerekçeyle

notification-api'den: sabit pencere → kayan pencere (Ay 4); time.Sleep retry döngüsü → kalıcı takvim (Ay 5); tek süreçte cache invalidasyonu → instance'lar arası invalidasyon (Ay 4); hardcoded sırlar → varsayılsız zorunlu env değişkenleri (Ay 1).

12. Riskler ve önlemler

Risk	Etki	Önlem
OT transform/compose hataları (Ay 2)	İki istemci sessizce farklı metinlere yakınsar; sonraki tüm aylar bozuk temele oturur	ot/ paketi ayın ilk ve tek işi; rastgele op üreticisiyle 10 000 iterasyonluk fuzzer; tie-break kuralı için açık test; fuzzer geçmeden başka işe başlanmaz
Sticky routing ihlali (Ay 3)	İki instance aynı dokümanı sıralar, günlük bozulur	Route başına strateji tablosu; (doc_id, version) primary key veritabanı düzeyi güvence; X-Doc-Instance ile gözlemlenebilirlik testi
Olay kaybı (Ay 5)	Webhook hiç gitmez, sessiz veri tutarsızlığı	Outbox aynı transaction'da; SKIP LOCKED relay; event_id idempotency; kill -9 senaryoları doğrulama bloğunda
Kimlik header sahteciliği	Bir kullanıcı başka kullanıcı gibi davranır	Trusted proxy istemci header'larını üzerine yazar ; backend'ler X-Gateway-Key ister; sabit zamanlı karşılaştırma; Ay 1 doğrulamasında açık test
Kapsam kayması (zengin metin, offline, servis keşfi)	Ay takvimi kayar	Bilinen ödünler listesi baştan yazıldı; her ay "kavram mı, ürün özelliği mi" sorusuyla filtrelenir
Aylık takvim baskısı	Doğrulama ve README güncellemesi atlanır	Ay sonu ritüeli teslim tanımıdır; doğrulama bloğu çalıştırılmadan ay kapanmaz
Yerel ortam çakışmaları	Üç proje aynı anda çalışmaz	Portlar baştan ayrıştırıldı (9000+, 3308, 6381); compose dosyasında belgelendi

13. Mevcut durum ve sonraki adımlar

16 Eylül 2026 itibarıyla (resmi başlangıç olan Ekim'den önce, hazırlık olarak):

- README (tek doğruluk kaynağı) tamamlandı: dokuz kavram, mimari, veri modeli, API yüzeyi, tam OT spesifikasyonu, altı aylık yol haritası, kurallar ve ödünler.
- **Ay 1 madde 1 tamamlandı:** go.work, docker-compose.yml (MySQL 3308, Redis 6381), db/gateway_schema.sql, db/doc_schema.sql, .gitignore. Container'lar healthy, şemalar yüklendi ve canlı volume'a tekrar uygulanabilirliği doğrulandı.
- **Ay 1 madde 2 tamamlandı:** gateway auth — /register, /login, /me, /admin/users, /health; JWT (HS256, 24 saat, user_id/role/exp); request_id ve logger middleware (loglarda yalnızca path); admin middleware; varsayılsız sırlar. Birim testleri (validation, jwt, admin, request_id, user service) — race ile yeşil; MySQL'e karşı uçtan uca curl doğrulaması yapıldı.

Sonraki adımlar (sırayla):

1. Ay 1 madde 3: internal/proxy/trusted_proxy.go portu ve /documents* reverse proxy'si.
2. Ay 1 madde 4: doc-service (gateway_auth, documents/members CRUD, admin, sayfalama).
3. Ay 1 madde 5-6: editor.html v1.
4. Ay 1 madde 7: yetki matrisi testleri; README Month 1 bölümünü "yapılan"a çevir; Month 1: ... commit'i.
5. Ay 2'ye doc-service/internal/ot/ ve fuzzer ile başla; fuzzer temiz geçmeden başka hiçbir şeye başlama.